

RegLens



AI Systems Architecture Review

Auditable regulatory RAG — cited answers, verified quotes, and a hash-chained trail a reviewer can replay, fully offline

Release/ship date · 12 September 2026 · Roshan Rana

Executive summary

What it is

- A regulatory question-answering system for compliance teams: hybrid retrieval, exact-citation routing, quote-verified grounded generation, and abstention when evidence is weak.
- Every query is written to a hash-chained audit record with an evidence digest, so a reviewer can replay what the system read before it answered.

What it proves

- Verification is code, not a prompt: citations must resolve to retrieved chunks and quotes must appear in the cited text, checked after generation, every time.
- 275 deterministic tests and an offline eval harness — including four adversarial source-instruction cases — run with no API key, no vector database, no model download.

Why it is enterprise-ready

- Fake embeddings, generation and reranking are real implementations bound by default; OpenAI, Qdrant and a cross-encoder reranker are optional layers that fail closed until explicitly configured.
- CI runs the full offline gate (lint, mypy, 275 tests, eval) plus a container-config check on every push.

Ask of the audience: agree the pilot scope for live OpenAI + Qdrant (cost, latency, model choice), the OCR opt-in path, and the production auth/rate-limit posture.

The problem and the users

The analyst's problem

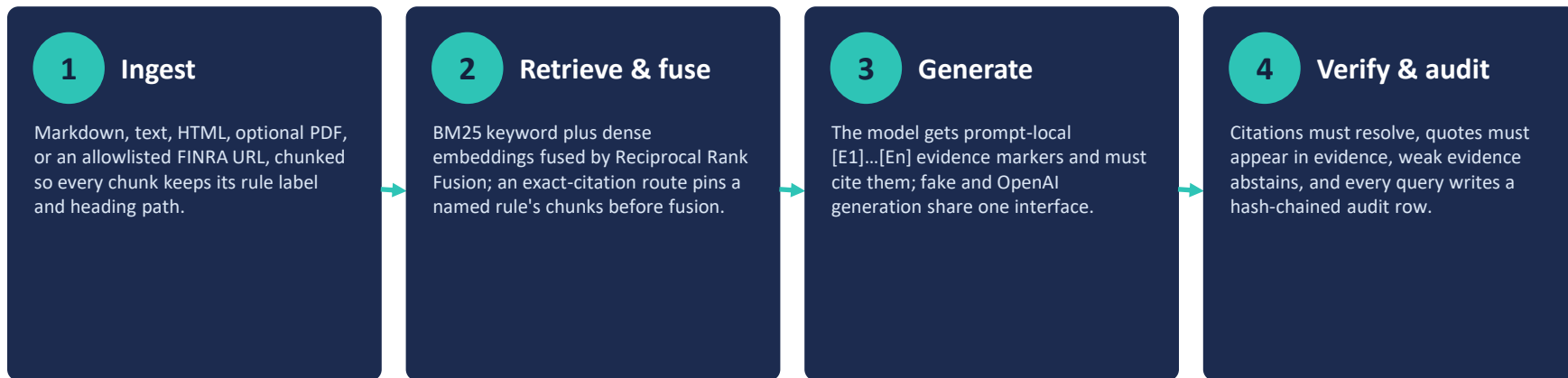
A compliance officer gets a question from the trading floor: can we do this? The answer lives in a rulebook, notices and internal policy. A general-purpose chatbot will retrieve roughly the right passage, paraphrase it, cite something, and answer even when the evidence is thin — and leave no record of what it read.

The firm's constraints

A wrong citation is a liability and an unsupported answer is worse than no answer. Anything the tool says may be relied on and later examined, so citations must be structural, evidence must be verified in code, and every answer needs a replayable audit trail.

Actor	Needs	Frequency
Compliance analyst (primary)	A cited answer with retrieval diagnostics, or an honest refusal when evidence is weak	Several times a day
Legal / policy / model-risk reviewer	Source-level traceability: what evidence, what model, what prompt version, replayable months later	On review
Engineering / platform reviewer	Deterministic offline tests and eval, fail-closed optional providers, one command to verify	Deploy / operate
Automation and AI assistants	The same grounded answers over HTTP: /query, /chat (with SSE streaming), audit export	Ad hoc

Solution at a glance



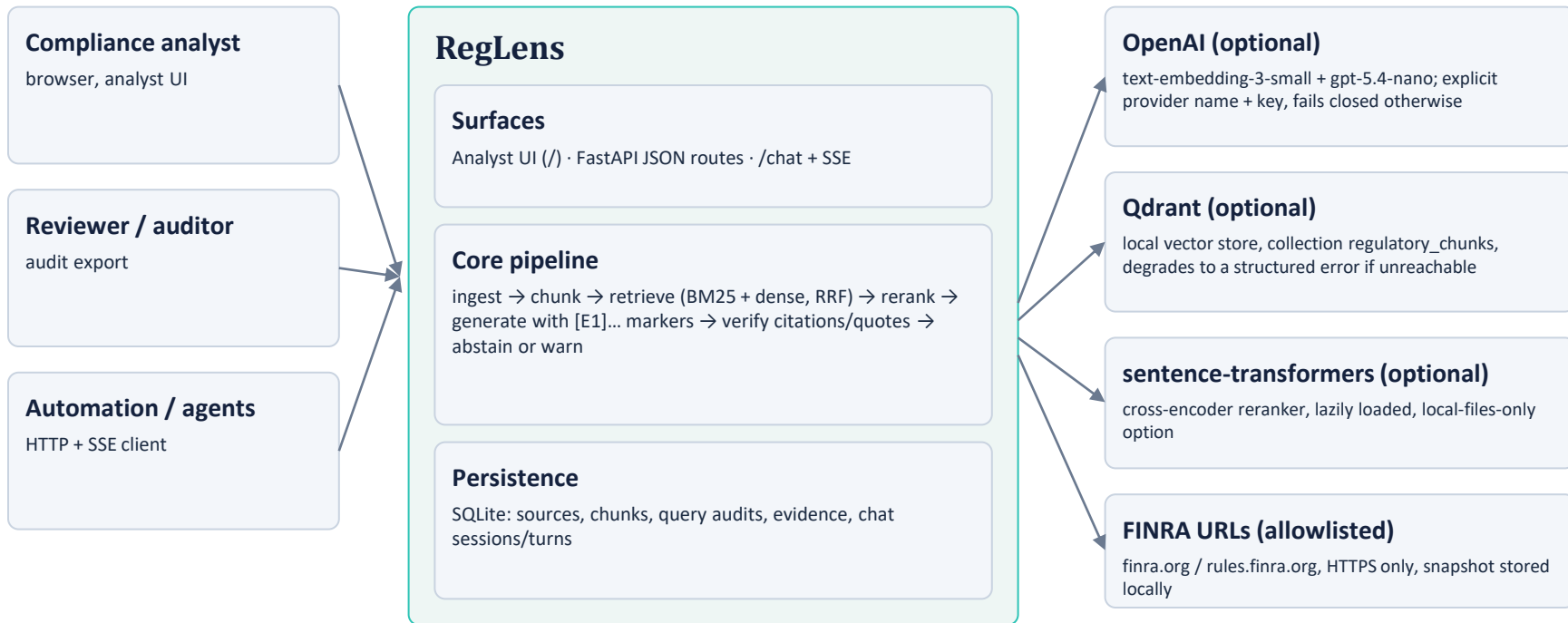
Surfaces

A dependency-free analyst UI served by FastAPI · JSON API (/retrieve, /query, /admin/ingest, /audit) · chat with optional Server-Sent Events streaming for app and agent workflows.

Two run modes, one contract

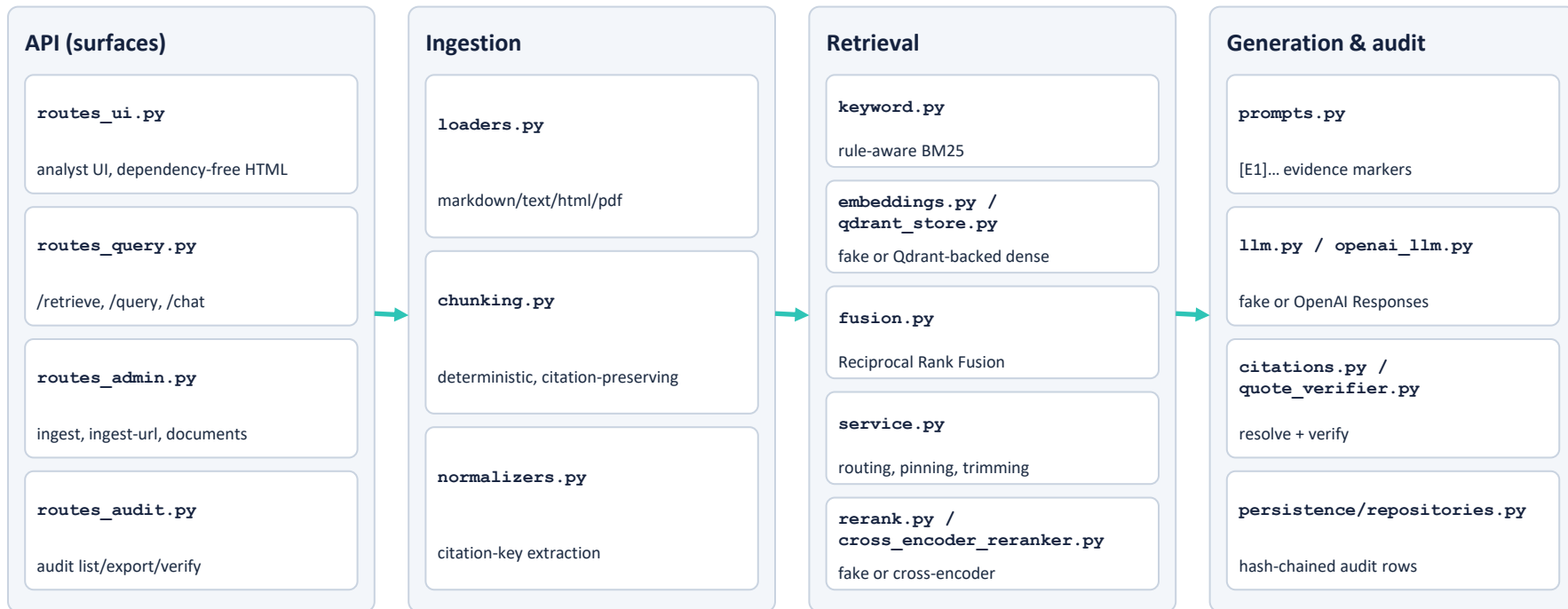
Mock mode (default): fake embeddings, generation and reranker, fully offline, no key, no Qdrant. Local mode: the same code path with Qdrant and, optionally, OpenAI and a cross-encoder reranker behind explicit configuration.

System context



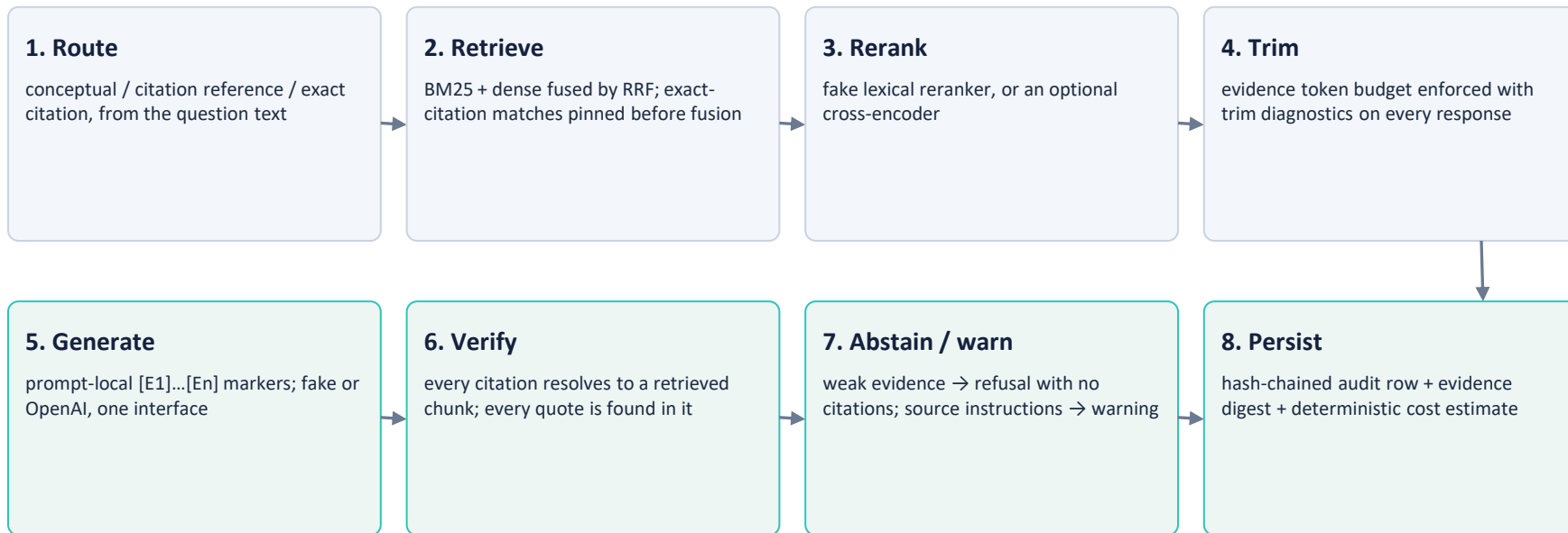
Mock mode (default) never calls any of the four boxes on the right. Local mode adds Qdrant; OpenAI and the cross-encoder are opt-in extras layered on top, each gated by an explicit setting.

Component architecture



Dependency direction is left to right. Every retrieval and generation provider is chosen by explicit settings, not by which package happens to be importable.

The answer path



Refusal is a first-class outcome, not an error path: a refusal is the only answer allowed to carry no citations, and the eval measures it (refusal accuracy 100.0%, 0 false abstentions).

Verify, don't trust: the citation and quote contract

What must be true for a non-refusal answer

- every cited [E1]...[En] marker must resolve to a chunk that was actually retrieved (citations.py)
- every quoted span must literally appear inside that chunk's text, case- and whitespace-normalised (quote_verifier.py)
- a non-refusal answer with no citations is rejected; a refusal is the only answer allowed to have none

Enforced by tests, not by prompt wording

```
test_verify_answer_citations_rejects_fabricated_evidence_id
test_verify_answer_citations_rejects_non_retrieved_chunk_citation
test_verify_answer_citations_rejects_quote_absent_from_evidence
test_non_refusal_answer_without_citations_is_rejected
```

✓ **verified**

quote found in the cited, retrieved chunk

✗ **rejected**

fabricated evidence id, or a quote not in the chunk

⚠ **warned**

source text tried to inject an instruction; fact kept, instruction dropped

◇ **abstained**

evidence too weak; refusal allowed to carry no citations

Why this matters

Citation faithfulness is the eval's headline KPI: 21/21 answers, every quote verified (100.0%).

Data architecture: stores, retention, classification

SQLite (metadata + audit)

sources, sections, chunks, query audits, query evidence, chat sessions and turns. Query audit rows are append-only: re-saving an existing query_id is rejected, not updated.

Vector store

In-memory fake embeddings by default (mock mode); an optional Qdrant collection regulatory_chunks in local mode, degrading to a structured error if unreachable.

Retention rules

- deleting a source cascades its sections/chunks and refreshes retrieval
- deleting a chat session removes history but keeps its query audits
- audits are immutable by design; chat history is not

Classification and provenance

Every chunk carries its source_id, checksum, corpus_id/version and heading path. Remote (FINRA URL) ingestion snapshots raw bytes locally under REGLENS_DOCUMENT_STORAGE_PATH before chunking, so provenance does not depend on the live page. reglens.db and reports/ are demo-local and gitignored; a production deployment swaps SQLite for a managed store without touching the audit-hash contract.

The bundled synthetic_rulebook.md fixture (11 sections/chunks) is what the eval and the screenshots in this deck run against; the schema is the same one a real rulebook would use.

Provider and live-mode design: fail closed by default

OpenAI (optional)

install `.[openai]`, set an explicit provider name and `OPENAI_API_KEY`; text-embedding-3-small + gpt-5.4-nano, capped output tokens, bounded embedding cache keyed by provider/model/dims/hash.

Qdrant (optional)

install `.[qdrant]`, `REGLENS_RAG_MODE=local`; a separate collection per embedding-dimension change; unreachable Qdrant degrades /ready, not startup.

Cross-encoder (optional)

install `.[rerank]`; ms-marco-MiniLM-L-6-v2, lazily loaded on first use; `REGLENS_CROSS_ENCODER_LOCAL_FILES_ONLY` avoids a network fetch.

The fail-closed test

Mock mode rejects live-provider flags outright; setting `OPENAI_API_KEY` alone enables nothing. ``test_openai_api_key_env_does_not_enable_live_providers_by_itself`` and ``test_provider_factories_do_not_import_openai_sdk_when_api_key_is_missing`` make the boundary a test, not a convention.

FINRA URL ingestion

POST `/admin/ingest-url` accepts only HTTPS `finra.org` / `www.finra.org` / `rules.finra.org`, validates the final redirect host, caps response bytes, snapshots raw bytes locally, then runs the normal ingest/chunk/audit path.

Every optional branch returns a structured `dependency_unavailable` error rather than crashing at startup when its package, key, quota or model is missing.

Design choices and why

Decision	Alternatives considered	Why this one
Verification in code after generation, not in the prompt	Trust the model to cite correctly; rely on prompt wording alone	A wrong citation in compliance is a liability the prompt cannot guarantee away; a code check can be tested and is enforced on every answer
Fake providers are real implementations, bound by default	Mocked test doubles that do not resemble production behaviour	275 tests, the eval harness and the UI all run offline with no key; the same interface later carries a live provider
Hybrid BM25 + dense fused by RRF; Qdrant optional	Dense-only retrieval; a vector database required from day one	Works fully in-memory in mock mode; Qdrant is additive for local/production scale, not a hard dependency
Prompt-local [E1]...[En] evidence markers	Free-form citations inside the generated answer text	Markers exist only in the prompt, so every citation the model produces must resolve to something the system actually retrieved
Only a refusal may lack citations	Allow any answer to omit citations when the model is unsure	Makes 'no evidence' and 'unsupported claim' distinguishable in code, not just in tone
OCR deferred, scanned PDFs fail closed	Best-effort OCR bundled into default PDF ingestion	Uncertain OCR text must not silently enter a compliance answer without an explicit confidence and audit policy (docs/ocr-strategy.md)
SQLite for metadata and audit	PostgreSQL from the first line of code	Zero-infrastructure local development and CI; the audit hash-chain is a code invariant, not tied to a specific store

Major features

Citation-preserving ingestion

Markdown, text, HTML, optional PDF with page numbers, plus allowlisted FINRA URL ingestion that snapshots raw bytes

Hybrid retrieval with exposed diagnostics

BM25 + dense fused by RRF, exact-citation pinning, optional reranking; every response carries dense/keyword/fusion scores

Grounded, verified generation

Prompt-local evidence markers; citation and quote verification in code after every generation call

Weak-evidence abstention and safety warnings

Refusal when evidence is thin; adversarial source-instruction text is filtered and warned on, never obeyed

Hash-chained audit trail

Append-only query audits with evidence digests; JSON/Markdown export; /audit/verify walks the chain

Durable chat with SSE streaming

/chat sessions persist in SQLite, link to immutable audits, and export as reviewer-friendly transcripts

A cited answer, verified against retrieved evidence

Answer

FINRA Rule 1030(b) states: Retention Period Records required by this rulebook must be retained for six years unless a longer period is required by applicable law. [E1]

Citations

FINRA Rule 1030(b) **verified**

Retention Period Records required by this rulebook must be retained for six years unless a longer period is required by applicable law.

Provenance

Session	cht_2bef2896a04db2eecd2c2b78b2628013d2b85af5
Turn	trn_6f62675bb6aa79c892cee0c0916fb5bfe65793c8
Query	qry_4c73d7ec79ab28af70ff980e369aef3d396f2392
Record	d39aedef8d6d7687e065b012591c94a88a0765214aed91caf7d9c54abae09c17
Evidence	3440a3f52eeb9d6bc5dfc1a83b2536f9da0402c164a45b7563fda78db0bac542

Evidence

1. FINRA Rule 1030(b) **0.0328**

FINRA Rule 1030(b). Rule 1030(b). Retention Period
Heading path: FINRA Synthetic Rulebook > Rule 1030.
Escalation and Recordkeeping > Rule 1030(b). Retention
Period Records required by this rulebook must be retained
for six years unless a longer period is required by
applicable law. Records must be searchable by customer,
account, rule, and date.

2. FINRA Rule 1040(a) **0.0323**

...mated Compliance Tools Heading path: FINRA Synthetic
Rulebook > Rule 1040. Governance and Model Use > Rule
1040(a). Automated Compliance Tools Members using
automated compliance tools must validate the tool before
production use and after material changes. Validation
evidence must include test cases, limitations, approval

"How long must records be retained?" against the bundled synthetic rulebook. FINRA Rule 1030(b) is cited, its quote is verified, and the provenance panel shows the audit hash chain and evidence digest.

Weak evidence produces a refusal, not a guess

Ask

QUESTION

What is the weather like in Miami this weekend?

SOURCE FILTER

All indexed sources

EVIDENCE COUNT

3

Ask

New chat

Refresh chats

Export chat

Verify audit

How long must records be retained? / cht_2bef2896a04db2eecd2c2b78b2628013d2b85af5

Answered with 0 citations.

Answer

I do not have sufficient retrieved evidence to answer this question.

weak_retrieval: Retrieved evidence was too weak to answer safely.

Citations

No citations returned.

Evidence

1. FINRA Rule 1030(b) 0.0323

FINRA Rule 1030(b). Rule 1030(b). Retention Period
Heading path: FINRA Synthetic Rulebook > Rule 1030.
Escalation and Recordkeeping > Rule 1030(b). Retention
Period Records required by this rulebook must be retained
for six years unless a longer period is required by
applicable law. Records must be searchable by customer,

What you are looking at

"What is the weather like in Miami this weekend?" retrieves only weakly related evidence; the system abstains with a `weak_retrieval` warning and zero citations, and the chat history shows both turns side by side.

The offline gate: tests, eval and the audit chain

```
$ python -m pytest -m "not live_openai and not requires_browser and not requires_qdrant and not requires_model_download"
..... [ 26%]
..... [ 52%]
..... [ 78%]
..... [100%]
275 passed, 5 deselected, 2 warnings in 8.50s

$ python -m scripts.run_evals --headline metrics/headline.json
{
  "passed": true,
  "summary": {
    "case_count": 21.0,
    "retrieval_recall_at_3": 1.0,
    "retrieval_recall_at_5": 1.0,
    "retrieval_recall_at_10": 1.0,
    "retrieval_mrr_at_10": 1.0,
    "citation_precision": 1.0,
    "quote_verification_rate": 1.0,
    "refusal_accuracy": 1.0,
    "answer_safety": 1.0,
    "warning_recall": 1.0,
    "audit_completeness": 1.0,
    "avg_latency_ms": 5.8
  }
}
headline written to metrics\headline.json

$ curl -s http://127.0.0.1:8604/audit/verify
{"verified": true, "record_count": 4, "latest_chain_index": 3, "failure_count": 0, "failures": []}
```

Commands run for this deck: the default pytest marker set (275 passed, 5 deselected), the eval harness (writes metrics/headline.json), and /audit/verify against the running server.

Every route is a typed, documented contract

RegLens 0.1.0 OAS 3.1
[/openapi.json](#)
RegLens API for cited regulatory question answering.

default		^
GET	/health Health	v
GET	/ready Ready	v
query		^
GET	/chat/sessions List Chat Sessions	v
GET	/chat/sessions/{session_id} Get Chat Session	v
DELETE	/chat/sessions/{session_id} Delete Chat Session	v
GET	/chat/sessions/{session_id}/export Export Chat Session	v
POST	/retrieve Retrieve	v
POST	/query Query	v
POST	/chat Chat	v
audit		^
GET	/audit/queries List Query Audits	v

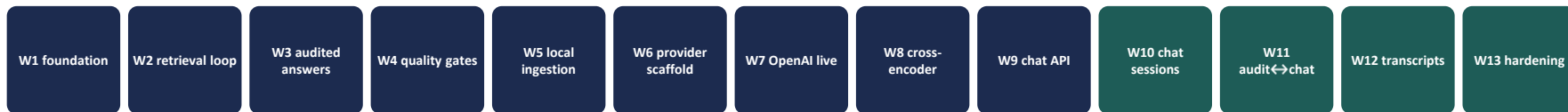
Auto-generated OpenAPI 3.1 documentation for the running service on port 8604: health/readiness, chat sessions, retrieval, query and audit routes.

Security and compliance controls

Boundary	Threat	Control in the code	Evidence
B1 user question	prompt injection through the question text	question length bound; routing and retrieval do not execute instructions found in input	eval safety cases
B2 source documents	instructions embedded inside ingested regulatory text	instruction clauses are filtered and warned on without dropping the factual clause in the same sentence	4 adversarial eval cases, warning_recall 100.0%
B3 API auth and rate limit	unauthenticated or unbounded access to operational routes	optional API-key auth (header or bearer) + per-minute rate limiting; health/ready/docs stay exempt	test_api_key_auth_protects_operational_routes_but_not_health
B4 remote ingestion	fetching or redirecting to an untrusted host	HTTPS-only, FINRA domain allowlist, byte-size cap, final-redirect host validated, snapshot stored locally	OPERATIONS.md ingest-url contract
B5 live providers	a stray key or flag silently enabling a live model	mock mode rejects live-provider flags outright; a key alone enables nothing	test_openai_api_key_env_does_not_enable_live_providers_by_itself
B6 audit integrity	edited or deleted evidence rows after the fact	append-only query audits, hash chain plus evidence digest, /audit/verify detects tampering	GET /audit/verify

Residual risks, stated: API-key auth and in-memory rate limiting are local controls, not an identity system; OCR for scanned PDFs is deferred and fails closed rather than guessing.

How it was built: 13 waves behind one gate



275

deterministic fake-mode tests, plus 5 deselected optional-profile tests

13

build waves, each shipped behind the same offline gate

21

eval fixture cases, including 4 adversarial source-instruction safety cases

2

CI jobs on every push: default-verify (lint, mypy, tests, eval, card-drift) and container-verify

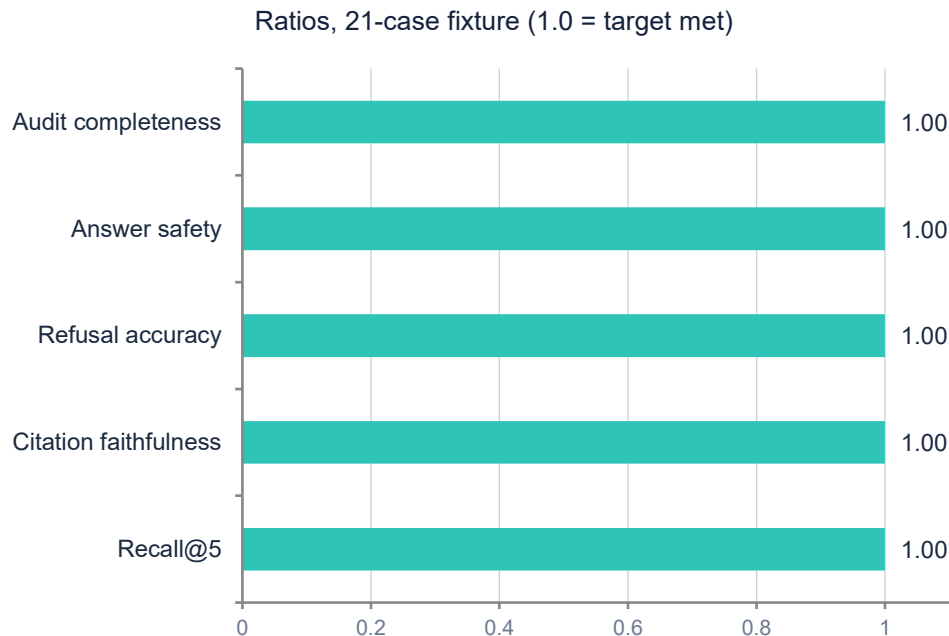
One command validates everything

``make verify`` = ruff lint + mypy strict + the default pytest marker set + the eval harness. CI runs the same profile via ``python -m scripts.verify default``, plus a README results-card drift check.

Deterministic-first, live optional

Fake embeddings, generation and reranker are real implementations satisfying the same interface as live providers, so the whole gate — UI included — never needs a key, a vector DB, or a model download.

Quality metrics from the gate



KPI	Value	How measured
Tests	275 passed, 5 deselected	pytest, this run
Abstention rate	14.3% (3/21), 0 false abstentions	make eval
Citation faithfulness	100.0% (21/21)	make eval
Recall@5 reranker ON vs OFF	100.0% vs 94.4% (n=18)	headline.json
Cost / query (fake-run model)	\$0.000202	costing model
Avg eval latency	5.8 ms (in-process, fake providers)	make eval
Live OpenAI cost / cross-encoder	pending	needs key/model

All five ratios read 1.0 on the 21-case offline fixture; the honest caveat is the corpus is 15 chunks and the embeddings are deterministic lexical hashes, not a production-scale semantic benchmark.

Quirks and known limitations (stated, not hidden)

100.0% recall/faithfulness is a small-fixture number

21 queries over a 15-chunk synthetic corpus with deterministic lexical embeddings; not a production-scale or semantic benchmark.

Live OpenAI cost is pending

The \$0.000202/query figure is the repo's char/4-token costing model over fake-generated text, not a metered API call.

Cross-encoder reranker is unmeasured

ms-marco-MiniLM-L-6-v2 needs sentence-transformers and a model weight download; the offline gate never fetches it.

OCR is a contract, not an implementation

Scanned/image PDFs return `corpus_load_error` by design; docs/ocr-strategy.md is a deferred, explicit decision.

Local controls, not an identity system

API-key auth and in-memory rate limiting are single-process; no SSO, no tenant isolation, no multi-instance limiter.

reglens.db, tmp/ and reports/ are gitignored

The demo database and reports regenerate from ``make eval`/ingestion`; this deck's screenshots came from a fresh run.

The bundled rulebook is synthetic

FINRA rule numbers and text in the fixture are illustrative for testing, not the live FINRA rulebook.

Container check is static

``scripts.verify container`` renders and validates Docker Compose config; this task did not build or run the image.

Cost, performance and operability

\$0.0002

mean fake-run cost per query, repo costing model, ~792 in / 34 out tokens

5.8 ms

average eval latency, in-process fake providers, 21 cases

21

fixture queries over a 15-chunk synthetic corpus, 2 corpora, seed 0

0 keys

needed for the full offline demo — mock mode, no OpenAI, no Qdrant

Operability

- /health and /ready with per-provider status, mode and gated errors
- `make verify` is the one-command gate; CI runs the same profile
- Docker + Compose default to mock-safe (no key, no Qdrant needed)
- Rollback is an environment variable: REGLENS_RAG_MODE mock ↔ local
- `scripts.verify container` statically checks the Compose config

Observability and audit

- Hash-chained query audit rows with an evidence digest per query
- GET /audit/verify walks the chain and reports failures, if any
- Request-ID propagation (X-Request-ID) through every response
- Structured dependency_unavailable errors, never a bare crash
- Deterministic cost estimate stored on every audit row

Roadmap to a pilot

Weeks 1–2

Harden data

- OCR as an explicit opt-in extra, behind `REGLENS_ENABLE_PDF_OCR`
- PostgreSQL for production metadata and audit storage
- Richer corpus version diffing

Weeks 3–4

Secure and scale

- SSO / RBAC in front of the API
- Tenant isolation for multi-firm deployments
- Durable async ingestion workers for larger corpora

Weeks 5–6

Measure on live

- A live OpenAI cost and quality run to replace the pending figures
- Cross-encoder reranker evaluated on a non-synthetic corpus
- Source-page citation highlighting in the UI

Week 7+

Operate

- OpenTelemetry / Prometheus-style observability
- A tamper-evident audit sink (external anchor for the hash chain)
- Managed deployment path for client infrastructure

Everything above is additive: the verifier, the abstention rule and the audit-chain contract do not change.

Appendix A — Decision log (abridged)

Area	Decision	Consequence
Grounding	Citations and quotes verified in code after generation, not asserted by the prompt	citations.py / quote_verifier.py; four rejection tests; citation faithfulness is a measured KPI
Refusal	Only a refusal may carry zero citations; any other answer without one is rejected	Weak evidence produces an honest abstention instead of a confident guess
Providers	Fake embeddings/LLM/reranker are real implementations bound by default	275 tests + eval + UI run with no key, no Qdrant, no model download
Providers	Live providers fail closed: a key alone never enables them	test_openai_api_key_env_does_not_enable_live_providers_by_itself
Retrieval	Hybrid BM25 + dense via RRF; exact-citation queries are pinned before fusion	A question naming a rule number always sees that rule's chunks first
Ingestion	Remote ingestion is HTTPS-only and allowlisted to FINRA domains, with a local snapshot	Provenance for remote sources does not depend on the live page staying reachable
Audit	Query audit rows are append-only; re-saving an existing query_id is rejected	The audit trail cannot be silently edited by a retry or a bug
Scope	OCR is a documented, deferred decision; scanned PDFs fail closed	No uncertain OCR text can enter a compliance answer without an explicit policy

Appendix B — Build statistics and lessons

Item	Value
Build waves (docs/CHANGELOG.md)	13, Wave 1 (foundation) through Wave 13 (production hardening)
Tests, default marker set	275 passed, 5 deselected (live_openai, requires_browser, requires_qdrant, requires_model_download)
Eval fixture	21 cases (14 answerable, 3 out-of-scope, 4 adversarial source-instruction safety) over 2 corpora
CI jobs	default-verify (lint, mypy, tests, eval, README card drift) and container-verify, on every push/PR
Optional profiles	test-browser, test-qdrant, test-models, test-container, verify-openai, verify-full-local
Lines of code (app/ + tests/)	≈ 19,600 across 123 files

Lessons the design carries forward

- Verification belongs in code that runs after generation, not in prompt instructions the model can drift away from.
- A fake provider must satisfy the real interface, or the tests it enables are not proving anything about production.
- An audit trail is only trustworthy if the store rejects edits at the boundary (append-only), not by convention.
- Optional heavy dependencies (Qdrant, OpenAI, cross-encoder) should fail closed with a structured error, never a crash.

Appendix C — Repository map and how to run

app/	FastAPI app: main.py, core (config, security, costing, errors), domain (ids, models)
app/api/	routes_ui, routes_query, routes_admin, routes_audit
app/ingestion/	loaders, chunking, normalizers
app/retrieval/	keyword (BM25), embeddings, qdrant_store, fusion, rerank, cross_encoder_reranker, service
app/generation/	prompts, llm, openai_llm, citations, quote_verifier, warnings, service
app/persistence/	db, repositories (SQLite: sources, audits, chat)
app/evals/	fixtures/, metrics.py, headline.py
tests/	275 default tests (unit, integration, e2e)
scripts/	run_evals.py, verify.py
metrics/	headline.json, render.py (README card, drift-checked)
docs/	OVERVIEW, SHOWCASE, OPERATIONS, CHANGELOG, ocr-strategy, project brief, this deck

Run it

```
python -m venv .venv
make install
make verify
python -m uvicorn app.main:app --reload

# ingest the bundled fixture
curl -X POST :8000/admin/ingest \
  -d '{"path":"app/evals/fixtures/..."}'

# optional: local mode with Qdrant
export REGLENS_RAG_MODE=local
make qdrant-up

# optional: live OpenAI
export OPENAI_API_KEY=<key>
make verify-openai
```

Questions

RegLens · roshanrana/RegLens · main branch · README, OVERVIEW, SHOWCASE and OPERATIONS in the repository